

>_ SESSION 03 • WEB DEVELOPMENT

CSS Fundamentals

Selectors, the box model, colors and units, typography basics, and how it all gets linked to your HTML.



>_ Vasken Sarkis • CSS Fundamentals

Today's session

- 01 Selectors & specificity — how CSS finds and prioritizes targets
- 02 The box model — content, padding, border, margin
- 03 Colors & units — writing values CSS understands
- 04 Typography basics — font, size, weight, line-height
- 05 Linking stylesheets — inline, internal, external



ALSO TODAY

Lab: style the "About Me" page. Assignment 1 — Personal Portfolio Page v1 — is assigned at the end of class.

01

Selectors & Specificity

How CSS finds the elements it should style — and who wins when two rules disagree.

Selectors target elements

- 01 Type selectors match a tag, everywhere it appears: `p`, `h1`, `nav`.
- 02 Class selectors match a `class` attribute and are reusable across elements: `.card`.
- 03 ID selectors match exactly one element on the page: `#hero`.



KEY IDEA

Classes are the workhorse selector — reusable, readable, and not too specific. Reach for an id only when you truly mean "one, unique element."

CSS	Meaning	Simple idea
<code>nav a</code>	inside	<code>a</code> anywhere inside <code>nav</code>
<code>.card > h2</code>	directly inside	<code>h2</code> must be directly inside <code>.card</code>
<code>h2 + p</code>	immediately after	<code>p</code> immediately after <code>h2</code>
<code>h2 ~ p</code>	after	any <code>p</code> after <code>h2</code>

```
1 nav a {  
2   color: navy;  
3 }  
4 .card > h2 {  
5   font-weight: 600;  
6 }  
7 h2 + p {  
8   margin-top: 8px;  
9 }  
10 h2 ~ p {  
11   color: gray;  
12 }
```

NOTE

Combinators describe the relationship between two selectors, not a new kind of selector.

descendant · space

child · >

adjacent · +

sibling · ~

Specificity has four levels



Higher wins, regardless of where the rule sits in the file. Specificity beats source order — source order only breaks ties within the same level.

When two rules collide

```
buttons.css

1 #cta {
2   background: teal;
3 }
4
5 .button {
6   background: gray;
7 }
8
9 /* <button class='button' id='cta'>
10   renders teal */
```



KEY IDEA

The element is a class and an id. The id rule has higher specificity, so teal wins — even though the class rule appears later.

Avoid !important

buttons.css

```
1 .button {  
2   background: gray !important;  
3 }  
4  
5 #cta {  
6   background: teal;  
7 }  
8  
9 /* !important now wins,  
10    ignoring specificity entirely */
```



GOTCHA

Once one rule uses !important, the only way to override it is another !important with higher specificity. It doesn't fix specificity problems — it escalates them.



TIP

Fix specificity fights by simplifying selectors, not by adding force.

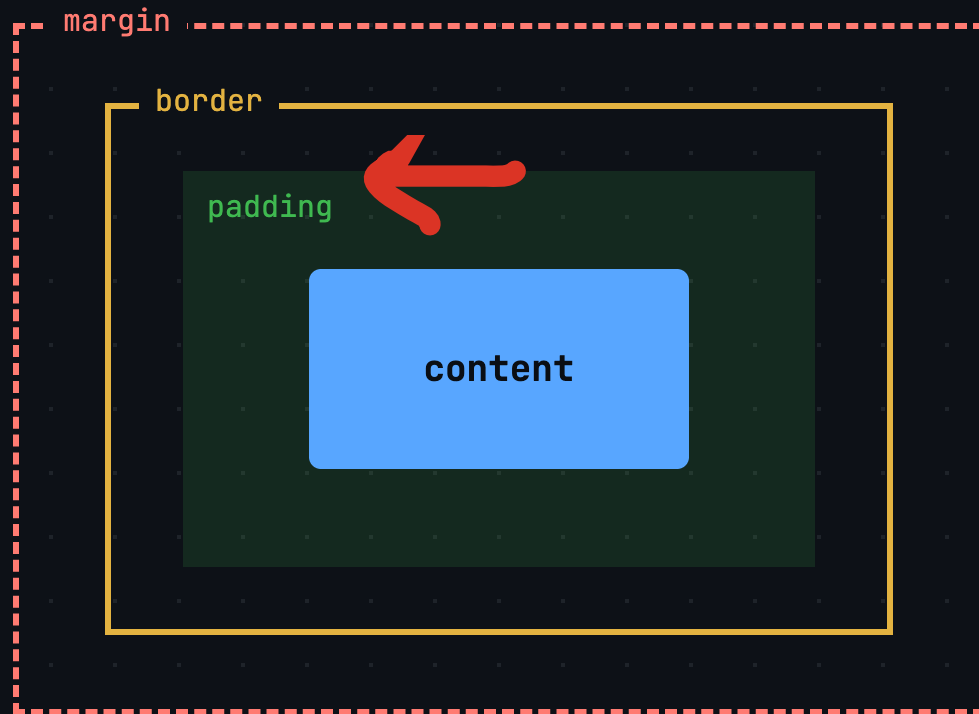
02

The Box Model

Content, padding, border, margin — four layers that decide the size and spacing of everything on the page.

>_ THE BOX MODEL

Every element is a box



content — the text, image, or element itself.

padding — space inside the border, part of the element's own background.

border — a visible (or invisible) line around the padding.

margin — transparent space that pushes other elements away.

>_ THE BOX MODEL

box-sizing changes the

box-sizing.css

```
1 .box {  
2   width: 300px;  
3   box-sizing: content-box;  
4 }  
5  
6 .box--fixed {  
7   box-sizing: border-box;  
8 }
```

	content-box	border-box
Content	300px	250px
Padding	40px	40px
Border	10px	10px
Total	350px	300px

By default, width sets the content box only — padding and border add on top of it.



TIP

box-sizing: border-box makes width include padding and border. Most projects set this globally.

Margins can collapse

cards.css

```
1 .card {  
2   margin-bottom: 40px;  
3 }  
4  
5 .card + .card {  
6   margin-top: 24px;  
7 }  
8 /* adjacent vertical margins collapse to  
9   the larger value: 40px, not 64px */
```



GOTCHA

Two vertical margins touching each other don't add up — the browser keeps only the larger one. Horizontal margins never collapse.



TIP

Padding never collapses. When spacing needs to be exact, padding is often the safer choice.



03

Colors & Units

Writing values CSS understands — and choosing ones that hold up when the screen changes size.

Three ways to write color

color.css

```
1 .a { color: #58a6ff; }
2 .b { color: rgb(88, 166, 255); }
3 .c { color: hsl(212, 100%, 67%); }
4
5 /* all three paint the exact same blue */
```

hex — 3 or 6 digits, most common

rgb — red, green, blue, 0-255

hsl — hue, saturation, lightness



TIP

hsl is the easiest to tweak by hand — lower the lightness for a darker shade, no calculator needed.

Absolute vs. relative units

px Fixed pixels

```
.box {  
  width: 320px;  
  font-size: 16px;  
}
```

- Same size on every screen
- Doesn't scale with zoom or root size
- Good for borders, shadows, fine details

rem / % Relative units

```
.box {  
  width: 80%;  
  font-size: 1.25rem;  
}
```

- Scales with the parent or root font size
- Adapts across screen sizes automatically
- The responsive default — reach for this first

04

Typography Basics

Font, size, weight, and line-height — the four properties that carry most of the reading experience.

Type properties, in order

type.css

```
1 body {  
2   font-family: 'Inter', sans-serif;  
3   font-size: 16px;  
4   font-weight: 400;  
5   line-height: 1.5;  
6 }  
7 h1 {  
8   font-size: 2.5rem;  
9   font-weight: 700;  
10 }
```

font-family

font-size

font-weight

line-height



NOTE

Always list a fallback: `system-ui`, `sans-serif` keeps text readable if the custom font hasn't loaded yet.



TIP

Write `line-height` unitless — it scales with `font-size` automatically.

Line-height and hierarchy

Aa h1 • 40px / 1.15

01 A tight line-height (1.1–1.2) reads well for large, short headings.

Aa h2 • 28px / 1.25

02 Body text needs more room — 1.4 to 1.6 keeps paragraphs from feeling cramped.

Aa body • 18px / 1.5

Aa caption • 14px / 1.4

03 Weight and size differences alone can create hierarchy — you rarely need color for that.

05

Linking Stylesheets

Three ways to attach CSS to HTML — and why only one of them belongs in a real project.

Three ways to add CSS

inline

```
<p style='color: teal;'>
  Hi
</p>
```

- Highest specificity — wins every tie
- No caching, no reuse across pages

internal

```
<style>
  p { color: teal; }
</style>
```

- Lives in the page's <head>
- Scoped to that one page only

external

```
<link rel='stylesheet'
      href='styles.css'>
```

- Shared across every page that links it
- Cached by the browser

External stylesheets win

- 01 One `styles.css` can style every page in a site — change it once, everything updates.
- 02 The browser caches it, so repeat visits load faster.
- 03 Structure (HTML) and presentation (CSS) stay in separate files — easier to read, easier to hand off.



KEY IDEA

Default to one external stylesheet, linked from the `<head>`. Reach for inline or internal only for a quick, throwaway test.

Style the "About Me" page

Take the bare "About Me" HTML from Session 2 and give it a real look.

1. Link one external `styles.css`
2. Set a color palette — background, text, one accent
3. Choose a font stack and size scale for headings and body
4. Add padding and margin so sections breathe

colors

fonts

spacing

Save often and reload in the browser to see changes.

Assignment 1: Personal Portfolio Page v1

HTML + CSS

static

single page

- Build on today's "About Me" styling — same page, taken further
- One HTML file, one linked external stylesheet
- Apply everything from this session: selectors, spacing, color, type



DUE

Session 5, at the start of class. Bring questions to the lab portions of Session 4 if anything's unclear.